certora

Security Assessment & Formal Verification Final Report

Saturn Dollar MO Extensions

March 2026

Prepared for Saturn

certora

Saturn

Table of Contents

certora Saturn

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Saturn Dollar MO Extensions	https://github.com/saturn-organization/saturn-yield-dollar https://github.com/saturn-organization/saturn-dollar	c5b19e5 (audit) 9c98678 (fix) 49240e5 (FV) e8d167c (audit) 11582ff (fix) 8735152 (FV)	Solidity

Project Overview

This document describes the security review of **Saturn Dollar MO Extensions**. The work was undertaken from **February 23rd, 2026** to **April 9th, 2026**.

The following contract list is included in our scope:

```
saturn-organization/saturn-yield-dollar/src/*
saturn-organization/saturn-dollar/src/*
```

The team performed a manual audit of all the files in scope**.** In addition, the Certora Prover demonstrated that the implementation of the Solidity contracts above is correct with respect to the formal rules written by the Certora team. During the audit and verification process, the Certora team discovered bugs in the Solidity contracts code, as listed on the following pages.

Protocol Overview

Saturn is a DeFi protocol that issues a USD-denominated stablecoin (USDat) and a yield-bearing vault token (sUSDat). USDat is built on M0's M token infrastructure as a JMIExtension, minted 1:1 against stable assets by KYC-verified users.

Users can permissionlessly deposit USDat into the sUSDat vault (an ERC4626-style contract) to gain synthetic exposure to STRC MicroStrategy's NASDAQ-listed preferred equity, which trades at approximately $100 and pays ~11% monthly cash dividends. STRC is tracked entirely on-chain via a virtual balance (`strcBalance`); no ERC20 token exists. A trusted entity buys and sells real STRC off-chain through a brokerage account and reconciles the on-chain state via `convertFromUsdat` and `convertFromStrc`, which are subject to oracle-validated tolerance checks.

STRC dividends are distributed to sUSDat holders through `transferInRewards` with linear vesting over a configurable period (default 30 days). The vault's `totalAssets()` is denominated in USD and reflects on-chain USDat , STRC balance corresponding to the STRC purchased by the offchain entity and the USD value of vested STRC priced via a Chainlink-compatible oracle.

Withdrawals from sUSDat are asynchronous and handled through a dedicated WithdrawalQueue implemented as an ERC721 contract. Users submit redemption requests that escrow shares into the queue and specify a minimum USDat to receive. The processor locks, processes, and fulfills requests by selling STRC off-chain and providing USDat. Processed requests can then be claimed by users. Requests cannot be cancelled, and compliance can seize blacklisted positions.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The primary security invariant is that `totalAssets() = usdatBalance + vestedStrcValue` must accurately reflect the protocol's backing at all times, and that shares burned during withdrawal processing correspond to value actually realized off-chain and reflected on-chain. Because settlement is delayed and execution occurs off-chain, correctness relies on enforcing these invariants through on-chain validation rather than atomic execution.

The most significant trust boundary is the PROCESSOR_ROLE, which supplies critical settlement parameters and controls conversion timing. While the protocol includes oracle checks, tolerance bounds (default 20%), vesting constraints, and validation logic to reduce reliance on processor honesty, the processor remains capable of influencing economic outcomes. Additional trusted roles include the Admin (pause, upgrades, parameters) and Compliance (blacklist, freeze, seize).

The audit considered attack vectors including:

- Decimal mismatches between USDat (6 decimals), vault shares (18 decimals), and oracle prices (8 decimals)
- Oracle failure scenarios, staleness during NASDAQ weekend closures, price bound breaches
- Processor under-settlement while satisfying tolerance checks
- ERC4626 inflation and donation attacks (mitigated by `_decimalsOffset = 12`)
- Withdrawal queue state machine edge cases , stuck requests, NFT transfer to blacklisted addresses
- Compliance bypass via blacklisted spender allowances

All core contracts were manually reviewed, including USDat, StakedUSDat, WithdrawalQueueERC721, and StrcPriceOracle. Invariants around share supply, NAV computation, and withdrawal settlement were analyzed across contract boundaries rather than in isolation.

The repository does not include test coverage for StakedUSDat or the associated withdrawal and processing modules. We recommend adding comprehensive tests covering all important scenarios to improve confidence in the protocol's correctness.

certora Saturn

FV Threat Coverage

This threat model covers the full Saturn protocol: the staking vault (**StakedUSDat**), NFT-based withdrawal queue (**WithdrawalQueueERC721**), STRC asset-price feed (**StrcPriceOracle**), and the USDat stablecoin (**USDat**). The analysis groups threats into four major risk categories, describing how each risk manifests, which components are affected, and which formally verified properties mitigate the threat.

1. Vault Share Price & Solvency Risks

Affected Components: StakedUSDat, StrcPriceOracle

Risk Description: StakedUSDat's share price is computed entirely from two internal accounting variables `usdatBalance` and `strcBalance` together with a live STRC price fed by the oracle. `totalAssets()` is the product of these values: any condition that causes it to underflow, overstate the vault's holdings, or silently shift after a governance action creates a direct path to either a complete denial of service or insolvency for depositors.

- **Vesting underflow DoS:** `_strcTotalAssets()` computes `(strcBalance - getUnvestedAmount()) × strcPrice`. If `getUnvestedAmount()` ever exceeds `strcBalance`, the subtraction underflows, causing every call to `totalAssets()` and by extension every deposit, mint, redemption, and conversion to revert. The entire vault is bricked until the invariant is restored externally.
- **USDat balance inflation:** StakedUSDat maintains a virtual `usdatBalance` that may diverge from the actual USDat token balance held by the contract. If `usdatBalance` exceeds the real on-chain balance, `totalAssets()` overstates the vault's holdings. Users redeeming shares receive USDat the vault does not actually hold, progressively draining it toward insolvency.
- **Unauthorized accounting writes:** Only a fixed set of functions should be permitted to modify `usdatBalance` or `strcBalance`. A bug that allows any other function to write to these variables whether by direct assignment or through an indirect call creates a vector for silent, undetected share-price manipulation that affects every outstanding share.
- **Admin-induced share-price shift via `setVestingPeriod`:** Changing the vesting period alters how much STRC is treated as unvested, which in turn changes the value returned by

`totalAssets()`. If a governance call to `setVestingPeriod()` caused an immediate reprice, a privileged actor could front-run the transaction and extract value from all existing depositors.

Relevant Properties:

- **Property-01:** `getUnvestedAmount()` never exceeds `strcBalance`, preventing underflow in `totalAssets()`.
- **Property-02:** `usdatBalance` never exceeds the actual USDat token balance held by the vault.
- **Property-06:** Only the explicitly permitted functions may modify `usdatBalance` or `strcBalance`.
- **Property-08:** Calling `setVestingPeriod()` does not change the value returned by `totalAssets()`.

Impact: A vesting invariant violation causes a **complete DoS** on the vault, that is no user can deposit, mint, or redeem until the state is corrected. A divergence in `usdatBalance` leads to **silent insolvency** that only surfaces at redemption time, potentially leaving a subset of depositors unable to recover their funds. An unauthorized write to either accounting variable, or a `setVestingPeriod` call that shifts `totalAssets()`, constitutes an admin-exploitable share-price manipulation affecting all outstanding shares.

2. Withdrawal Queue: STRC Burn Integrity, Slippage Guarantee & Liveness

Affected Components: WithdrawalQueueERC721, StakedUSDat

Risk Description: `processRequests()` is the most complex and highest-stakes operation in the protocol. It simultaneously burns STRC shares on StakedUSDat, allocates USDat pro-rata across multiple pending withdrawal requests, and finalises request statuses. A bug anywhere in this flow can either permanently lock user funds (by making `claim()` revert), allow the processor to pass an excessive STRC burn that leaves the vault under-collateralised, or leave the queue unable to pay all of its obligations.

- **Excessive STRC burn:** If the STRC amount passed to `burnQueuedShares()` exceeds the vested balance, the subtraction inside the vault underflows, reverting the entire

`processRequests()` call and locking all requests in the batch in a permanently non-claimable state.

- **Slippage floor manipulation:** A user sets a `minUsdatReceived` floor when requesting redemption. If a third party can raise this value after submission, the allocated `usdatOwed` will fall below the modified minimum, making `claim()` succeed but paying out less than the user expected. Conversely, if a third party can lower it, the user's slippage protection is silently removed. Either direction is a loss for the user.
- **Queue insolvency:** Once a request is marked Processed, the queue contract must hold sufficient USDat to pay every claimant. Any code path that marks requests as Processed without transferring the corresponding USDat would leave some claims permanently unredeemable.
- **NFT enumeration corruption:** The ERC721Enumerable index structures must remain consistent after every mint and burn. A stale or duplicate index entry causes `_removeTokenFromAllTokensEnumeration()` to revert inside `claim()`, permanently locking a processed withdrawal request even when USDat is available to pay it.
- **Claim liveness failure:** An eligible, non-blacklisted, non-paused owner of a Processed request must always be able to claim. Any combination of state invariant violations that causes `claim()` or `claimBatch()` to revert for a valid claimant constitutes a permanent loss of access to the user's funds.

Relevant Properties:

- **Property-03:** The STRC amount burned during `processRequests()` never exceeds the vested balance.
- **Property-04:** The slippage floor is faithfully stored at request time, can only be lowered by the token owner, every Processed request satisfies `usdatOwed >= minUsdatReceived`, the ERC721 structures remain consistent, and an eligible claimant can always successfully call `claim()`.
- **Property-05:** The queue's USDat balance always covers the total owed across all Processed requests.

Impact: Direct user-fund loss at the point of redemption, the highest-frequency, highest-value operation for vault participants. A broken slippage guarantee means users are underpaid relative to their stated tolerance. An excessive STRC burn or NFT enumeration corruption locks withdrawal batches permanently. Queue insolvency means some claimants cannot recover their deposited assets even after their request is marked Processed.

3. USDat Backing & Virtual Asset Accounting Risks

Affected Components: USDat

Risk Description: USDat is a non-rebasing ERC20 stablecoin backed by M0 assets and other approved collateral. It maintains virtual internal accounting that tracks the total value of assets deposited through each approved asset type. Two related failure modes exist: (i) the stablecoin becomes under-backed, meaning the total real collateral held is less than the supply of USDat tokens, so some holders cannot recover their assets on redemption; (ii) the virtual accounting overstates deposits, meaning the contract believes it holds more collateral than was actually transferred, allowing USDat to be issued against assets that were never received.

- **Under-backed USDat:** If at any point the total real assets held by USDat (across all backing types: M0 and allowed non-M assets) fall below the outstanding USDat supply, then at least one holder cannot redeem their tokens at par value. This can arise from a bug in the deposit or withdrawal logic, from an inconsistency between the virtual accounting and actual on-chain balances, or from an asset whose cap was set too permissively.
- **Virtual accounting overstatement:** USDat tracks deposits and withdrawals using internal counters separate from actual ERC20 `balanceOf` calls. If these counters can be inflated without a corresponding asset transfer, for example, through a re-entrant call, a fee-on-transfer asset discrepancy, or an unguarded accounting update, the contract issues USDat backed by assets it never received, immediately eroding the backing ratio.

Relevant Properties:

- **Property-09:** Total on-chain assets held by USDat always equal or exceed the outstanding supply, ensuring every token is redeemable at par.
- **Property-10:** Virtual accounting variables accurately reflect actual asset transfers; USDat cannot be issued without a corresponding asset receipt.

Impact: A backing shortfall means **some USDat holders cannot redeem at par**, that is a stablecoin depeg in the most direct sense. A virtual accounting overstatement creates **unbacked USDat supply** that silently dilutes the backing ratio for all holders, potentially precipitating a bank-run dynamic where later redeemers find insufficient collateral.

4. Compliance & Access Control Risks

Affected Components: StakedUSDat, WithdrawalQueueERC721, USDat

Risk Description: Saturn includes on-chain compliance controls across all three core contracts: addresses can be blacklisted or frozen, and any contract can be paused by the COMPLIANCE_ROLE. These controls must be uniformly enforced across every user-callable entry point. A gap in enforcement allows a sanctioned address to continue moving assets or claiming funds, and an ineffective pause leaves the protocol unable to halt an active exploit. The USDat contract adds a whitelist layer: only whitelisted addresses may wrap or unwrap.

- **Blacklist bypass on StakedUSDat:** A blacklisted user should not be able to deposit, mint, transfer shares, or interact with the vault in any way. Any function that omits the blacklist check becomes a bypass vector, allowing a sanctioned address to continue transacting.
- **Blacklist bypass on the WithdrawalQueue:** A blacklisted address should not be able to transfer its withdrawal NFT or claim USDat. Because the queue checks both `isBlacklisted` (StakedUSDat) and `isFrozen` (USDat), the absence of either check is sufficient to bypass the control.
- **Pause bypass on StakedUSDat:** When paused, all asset-moving operations must revert. The one intentional exception, `claim()` and `claimBatch()` remain callable to avoid permanently locking already-processed funds, must be tightly scoped: these functions must not permit the caller to also change their sUSDat share balance.
- **Pause bypass on the WithdrawalQueue:** When paused, all queue operations that move NFTs or USDat must revert. Only approval operations (which do not move assets) are permitted.
- **Whitelist and freeze bypass on USDat:** Only whitelisted addresses may call `wrap` or `unwrap`. Frozen addresses must be blocked from transferring USDat. Any function that omits these checks allows non-compliant actors to interact with the stablecoin.
- **Pause bypass on USDat:** When USDat is paused, all token-moving operations must revert. A bypass allows asset movements during an emergency halt, undermining the protocol's incident-response capability.

Relevant Properties:

- **Property-07:** Blacklisted addresses are blocked from all restricted functions on StakedUSDat and the WithdrawalQueue; pausing either contract halts all asset-moving operations; `claim()` under pause does not alter the caller's share balance.

- **Property-11:** Whitelist, freeze, and pause controls are uniformly enforced across every user-callable entry point on USDat.

Impact: Compliance failures expose the protocol to regulatory and reputational risk, and may allow sanctioned actors to extract or transfer funds that should be seized or frozen. An ineffective pause undermines the protocol's primary emergency-response mechanism, potentially allowing an exploit to continue draining funds after the compliance team has attempted to halt the system. A whitelist or freeze bypass on USDat directly violates the M0 compliance requirements that underpin the stablecoin's legal standing.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	2	2	2
Medium	1	1	1
Low	3	3	3
Informational	4	4	3
Total	**10**	**10**	**9**

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

certora

Saturn

Detailed Findings

ID	Title	Severity	Status
H-01	Supply cap check uses mismatched decimals (can incorrectly block wraps or make cap meaningless)	High	Fixed
H-02	MIN_WITHDRAWAL threshold is incompatible with USDat decimals, permanently blocking all redemptions	High	Fixed
M-01	Supply Cap Can Be Bypassed Via `claimYield()` Since Yield Minting Ignores Supply-cap Checks)	Medium	Fixed
L-01	Whitelist checks are bypassed for JMI asset wraps	Low	Fixed
L-02	WithdrawalQueueERC721.addRequest uses _mint instead of _safeMint	Low	Fixed
L-03	getUnvestedAmount() <= strcBalance can be violated by a call to setVestingPeriod()	Low	Fixed
I-01	Compliance role cannot transfer its own NFTs	Informational	Fixed
I-02	Hardcoded price decimals in validation functions	Informational	Fixed

I-03	Paused USDat Could Lead To Unbacked `USDat` in the Staking Contract When `STRC` Is Purchased	Informational	Acknowledged
I-04	Redundant zero-address validation in USDat.initialize	Informational	Fixed

High Severity Issues

H-01: Supply cap check uses mismatched decimals (can incorrectly block wraps or make cap meaningless)

Severity: High	Impact: Medium	Likelihood: High
Files: saturn-dollar/USDat	Status: Fixed	

Description:

`USDat` enforces the supply cap in `_revertIfSupplyCapExceeded(amount)` by checking:

```
function _revertIfSupplyCapExceeded(uint256 amount) internal view {
        SupplyStorage storage $ = _getSupplyStorage();
        if ($.isEnabled && totalSupply() + amount > $.cap) {
            revert SupplyCapExceeded(totalSupply(), amount, $.cap);
        }
    }
```

`totalSupply()` is denominated in USDat (extension) decimals which are 6 and amount is passed into `_beforeWrap(account, recipient, amount)` by M0, and for JMI wraps this amount is the input asset amount, denominated in the asset's own decimals which can be 18 decimals for assets like DAI.

So the cap check is comparing values in different units. It treats the input amount as if it were already USDat units.

Due to this enabling the supply cap can incorrectly block wraps (e.g., wrapping 1 DAI reverts) because the cap is checked in 6-decimal USDat units while the wrapped asset amount is 18-decimals.

Recommendations:

Ensure the cap and asset amount is represented in the same decimal units.

Customer Response:

Fixed in 4e750b3.

Fix Review:

Confirmed.

H-02: MIN_WITHDRAWAL threshold is incompatible with USDat decimals, permanently blocking all redemptions

Severity: High	Impact: High	Likelihood: High
Files: saturn-yield-dollar/src/StakedUSDat.sol	Status: Fixed	

Description:

`StakedUSDat` defines a minimum withdrawal constant of `10e18`:

```
/// @notice Minimum withdrawal amount (10 USDat)
uint256 public constant MIN_WITHDRAWAL = 10e18;
```

Users withdraw from `StakedUSDat` by calling `requestRedeem(shares)`, which computes the corresponding asset amount via `previewRedeem(shares)` and passes it to `_processWithdrawal`, which function enforces a minimum:

```
function _processWithdrawal(...) internal ... returns (uint256 requestId) {
    ...
    require(assets >= MIN_WITHDRAWAL, WithdrawalTooSmall());
    ...
}
```

However, `previewRedeem` uses the ERC4626 `_convertToAssets` formula, which returns values denominated in the underlying asset's decimals. USDat uses 6 decimals, so `previewRedeem` always returns a 6-decimal value. For a deposit of 1,000,000 USDat (`1_000_000e6`), redeeming all resulting shares returns `assets ≈ 1e12` — six orders of magnitude below `MIN_WITHDRAWAL = 10e18`.

Note that `withdraw()` and `redeem()` are overridden to unconditionally revert, making `requestRedeem()` the sole withdrawal mechanism. With this path blocked, all deposited USDat is permanently locked in the vault.

All user funds deposited into `StakedUSDat` are irretrievable. Any call to `requestRedeem` will revert regardless of the number of shares being redeemed. Since this is the only withdrawal path available to users, the vault becomes a one-way deposit with no exit.

Recommendations:

Change `MIN_WITHDRAWAL` to a 6-decimal value consistent with USDat's precision:

```
- uint256 public constant MIN_WITHDRAWAL = 10e18;

+ uint256 public constant MIN_WITHDRAWAL = 10e6;
```

Customer Response:

Fixed in cbd0450.

Fix Review:

Confirmed.

Medium Severity Issues

M-01: Supply Cap Can Be Bypassed Via claimYield() Since Yield Minting Ignores Supply-cap Checks

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: saturn-dollar/USDat.sol	Status: Fixed	

Description:

The cap enforcement is applied in the wrapping path (`_beforeWrap()` → `_revertIfSupplyCapExceeded()`). However, in the M0 MYieldToOne design, `claimYield()` mints USDat directly to `yieldRecipient` using `_mint()` and does not go through `_beforeWrap()` .

So even when the supply cap is enabled, calling `claimYield()` can increase total USDat supply beyond the configured cap. This can lead to `totalSupply()` being greater than the supply cap , once `totalSupply()` exceeds the cap, all future wraps will revert.

Recommendations:

Enforce the supply cap on yield minting too (e.g., add a cap check in the `claimYield()` path via an override like `_beforeClaimYield()`)

Customer Response:

Fixed in 4e750b3.

Fix Review:

Confirmed.

Low Severity Issues

L-01: Whitelist checks are bypassed for JMI asset wraps

Severity: Low	Impact: Medium	Likelihood: Medium
Files: saturn-dollar/USDat	Status: Fixed	

Description:

`USDat` only overrides the 3-parameter `_beforeWrap(address account, address recipient, uint256 amount)` hook, but JMI asset wraps use JMIExtension implementation's `_beforeWrap(address asset, address account, address recipient, uint256 amount)`. During `wrap(asset, recipient, amount)`, the 4-parameter JMI hook is executed and its `super._beforeWrap(account, recipient, amount)` resolves to `MYieldToOne`, not USDat's 3-parameter override. As a result, USDat's whitelist checks are not enforced for external-asset wraps via the SwapFacility.

Recommendations:

Also override the 4-parameter JMI hook in `USDat` and enforce whitelist checks there.

Customer Response:

Fixed in 984f226.

Fix Review:

Confirmed.

L-02: WithdrawalQueueERC721.addRequest uses _mint instead of _safeMint

Severity: Low	Impact: Low	Likelihood: Low
Files: saturn-yield-dollar/src/WithdrawalQueueERC721.sol	Status: Fixed	

Description:

In `WithdrawalQueueERC721.addRequest`, the withdrawal request NFT is minted using `_mint`:

```
    function addRequest(address user, uint256 shares, uint256 minUsdatReceived)
        external
        nonReentrant
        whenNotPaused
        onlyRole(STAKED_USDAT_ROLE)
        returns (uint256 tokenId)
    {
        ...
        _mint(user, tokenId);

        emit WithdrawalRequested(tokenId, user, shares, block.timestamp);
    }
```

Unlike `_safeMint`, `_mint` does not call `onERC721Received` on the recipient. If a `user` is a contract that does not implement the `IERC721Receiver` interface, the NFT will be minted successfully, but the contract will have no way to interact with it. Since the withdrawal NFT represents escrowed sUSDat shares and is required to claim the underlying USDat, this would result in the withdrawal being permanently stuck.

The risk is limited because `requestRedeem` in `StakedUSDat` passes `msg.sender` as the user, and a contract initiating the redemption is likely aware of ERC721 interactions. However, the ERC721 standard recommends `_safeMint` to protect against accidental token loss in contracts that lack ERC721 support.

Recommendations:

Replace `_mint` with `_safeMint` to ensure the recipient can handle ERC721 tokens.

Customer Response:

Fixed in ceed44e.

Fix Review:

Confirmed.

L-03: `getUnvestedAmount() <= strcBalance` can be violated by a call to `setVestingPeriod()`.

Severity: Low	Impact: High	Likelihood: Low
Files: saturn-yield-dollar/StakeUSDat.sol	Status: Fixed	

Description:

If the processor increases the vesting period, the new period may reactivate the previously finished period. Such a situation increases the `unvestedAmount` without increasing the `strcBalance`, and may lead to the invalidation of `strcBalance >= getUnvestedAmount()`.

Recommendations:

In the function `setVestingPeriod()`, add `vestingAmount=0`.

Customer Response:

Fixed in 49240e5.

Fix Review: Confirmed.

Informational Severity Issues

I-01: Compliance role cannot transfer its own NFTs

Files:

saturn-yield-dollar/src/WithdrawalQueueERC721.sol

Description:

The `_updat` override in `WithdrawalQueueERC721` assumes any transfer by a `COMPLIANCE_ROLE` holder is a seizure, enforcing `_requireBlacklisted(from)`. This prevents compliance officers from transferring their own legitimately owned NFTs, since they are not blacklisted.

Recommendations:

Only enforce the blacklist seizure check when `from != msg.sender`.

Customer Response:

Fixed in d451cd3.

Fix Review:

Confirmed.

I-02: Hardcoded price decimals in validation functions

Files:

saturn-yield-dollar/src/WithdrawalQueueERC721.sol
saturn-yield-dollar/src/StakedUSDat.sol

Description:

`_validateConversion` (`StakedUSDat`) and `_validateTotals` (`WithdrawalQueueERC721`) hardcode price precision to `1e8`, while `_strcTotalAssets` dynamically fetches `priceDecimals` from the oracle. If `updateOracle()` switches to a feed with different precision, `totalAssets` calculation adapts, but the validation logic silently breaks.

Recommendations:

Replace hardcoded `1e8` with `10 ** priceDecimals` fetched from the oracle.

Customer Response:

Fixed in 3a6398b.

Fix Review:

Confirmed.

I-03: Paused USDat Could Lead To Unbacked `USDat` in the Staking Contract When `STRC` Is Purchased

Files:

saturn-yield-dollar/StakedUSDat

Description:

In a scenario where the offchain entity purchases `STRC` from the market the `PROCESSOR_ROLE` is supposed to call the `convertFromUsdat()` to account for the `STRC` purchased , but if the `UDSat` contract is paused i.e. the JMI extension token contract is paused then `USDat` transfers would also be paused i.e. would revert , this could lead to a situation where the `STRC` is purchased by the entity from the market but `USDat` supply can not be decreased synchronously from the Staking contract (since the call to `convertFromUsdat()` would revert) which would lead to unbacked `USDat` in the staking contract corresponding to the `STRC` purchased from the market till the `USDat` contract is resumed back again.

Recommendations:

Consider acknowledging the risk and make sure to not purchase `STRC` when `USDat` transfers are paused.

Customer Response:

Acknowledged.

I-04: Redundant zero-address validation in USDat.initialize

Files:

saturn-dollar/src/USDat.sol

Description:

`USDat.initialize()` performs an upfront zero-address check on all four parameters:

```
    function initialize(address yieldRecipient, address admin, address
compliance, address processor)
        public
        initializer
    {
        if (yieldRecipient == address(0) || admin == address(0) || compliance ==
address(0) || processor == address(0))
        {
            revert ZeroAddress();
        }
      ...
```

This check is redundant, as all four addresses are individually validated downstream during the initialization chain:

- `yieldRecipient` — validated in `MYieldToOne._setYieldRecipient`
- `admin` — validated in `MYieldToOne.__MYieldToOne_init`
- `compliance` — validated as `freezeManager` in `Freezable`and as `pauser` in `Pausable`
- `processor` — validated as `assetCapManager` in `JMIExtension` and as `yieldRecipientManager` in `MYieldToOne`

The check increases deployment gas cost without adding any safety benefit.

Recommendations:

Remove the redundant zero-address check from `USDat.initialize()` and rely on the existing downstream validations.

Customer Response:

Fixed in 18543f7.

Fix Review:

Confirmed.

Formal Verification

Verification Methodology

We performed verification of the **Saturn** protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the Certora Verification Language (CVL) and **Saturn**'s implementation source code written in Solidity.
More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

Formal Verification Properties Overview

ID	Title	Impact in case of violation	Status
P-01	getUnvestedAmount() <= strcBalance	Covers against risk R-1.	Verified
P-02	usdatBalance <= USDat.balanceOf(stakedUSDat)	Covers against risk R-1.	Verified
P-03	Processor cannot burn more STRC than exists in StakedUSDat	Covers against risk R-2.	Verified
P-04	users always receives at least the minimum USDat they specified	Covers against risk R-2.	Verified
P-05	The withdrawal queue always holds enough USDat to cover all processed requests	Covers against risk R-2.	Verified
P-06	Only the expected functions may modify usdatBalance / strcBalance	Covers against risk R-1.	Verified
P-07	Access Control for StakedUSDat and WithdrawalQueueERC721	Covers against risk R-4.	Verified
P-08	totalAssets() is unchanged after setVestingPeriod()	Covers against risk R-1.	Verified
P-09	USDat is always backed	Covers against risk R-3.	Verified

P-10	Virtual accounting is consistent and conservative	Covers against risk R-3.	Verified
P-11	Whitelist, freeze, and pause configurations are never ignored	Covers against risk R-4.	Verified

Formal Verification Properties

StakedUSDat and WithdrawalQueueERC721 Properties

Trust Assumptions

1. We assume that loop_iter is 3. That means that we only unroll Solidity loops up-to 3 times.

P-01. getUnvestedAmount() <= strcBalance	
Status: Verified	**High-level description** The unvested STRC amount never exceeds strcBalance.

Rule Name	Status	Description	Link to rule report
unvested_never_exceeds_strcBalance	Violated	*See L-03.* *Note: fixed in current version*	*Report*

P-02. usdatBalance <= USDat.balanceOf(stakedUSDat)	
Status: Verified	**High-level description** usdatBalance <= USDat.balanceOf(stakedUSDat)

Rule Name	Status	Description	Link to rule report
usdatBalance_leq_real_balance	Verified	*See above.*	*Report*

P-03. Processor cannot burn more STRC than exists in StakedUSDat

Status: Verified	**High-level description** The amount of STRC burned in a single processRequests() call never exceeds the contract's total internal STRC balance.

Rule Name	Status	Description	Link to rule report
processRequests_strc_amount_bounded	Verified	*When processRequests() calls burnQueuedShares(), the STRC amount passed does not exceed the vested STRC amount.*	*Report*
burnQueuedShares_access_control	Verified	*A call to burnQueuedShares from any address other than the WithdrawalQueue contract reverts.*	*Report*
only_processRequests_calls_burnQueuedShares	Verified	*Among all functions on the WithdrawalQueue, only processRequests() may result in a call to burnQueuedShares() on StakedUSDat.*	*Report*

P-04. users always receives at least the minimum USDat they specified

Status: Verified	**High-level description** When a user submits a withdrawal request, the minimum USDat amount they specify is recorded and preserved. Only the user can lower that minimum, and once the request is processed, the user is guaranteed to receive at least that amount upon claiming.	

Rule Name	Status	Description	Link to rule report
claim_succeeds_when_eligible	Verified	*An eligible, non-blacklisted, non-paused owner of a Processed request can always call claim() without reverting and receives at least minUsdatReceived.*	*Report*
claimBatch_succeeds_when_eligible_single	Verified	*An eligible, non-blacklisted, non-paused owner of a single Processed request can always call claimBatch() without reverting and receives at least minUsdatReceived.* ***Limitation**: this rule assumes a batch with a single request.*	*Report*
claimBatch_succeeds_when_eligible_two_batch	Verified	*An eligible, non-blacklisted, non-paused owner of two Processed requests can always call claimBatch() without reverting and receives at least minUsdatReceived for each request.* ***Limitation**: this rule assumes a batch with two requests.*	*Report*
only_owner_can_update MinUsdatReceived	Verified	*updateMinUsdatReceived reverts for any caller who is neither the token owner nor an approved operator for that token.*	*Report*
minUsdatReceived_only_changes_via_update_min	Verified	*The minUsdatReceived field of an existing request is never modified by any function other than updateMinUsdatReceived().*	*Report*

update_min_in_progress_only_decreases	Verified	*Calling updateMinUsdatReceived() on a request in the InProgress state can only lower the recorded minimum, never raise it.*	*Report*
addRequest_stores_minUsdatReceived	Verified	*When addRequest() is called, the minUsdatReceived parameter is stored in the newly created request.*	*Report*
minted_token_in_all_tokens	Verified	*An invariant helper:* *A token with a non-NULL status exists in _allTokens at its recorded index, and every slot in _allTokens maps back to a unique token — there are no duplicate entries.*	*Report*
nonexistent_token_has_no_approval	Verified	*An invariant helper:* *A token that has not been minted has no per-token approval recorded in the WithdrawalQueue.*	*Report*
no_approval_for_zero_address	Verified	*An invariant helper:* *The zero address is never recorded as an approved-for-all operator in the WithdrawalQueue.*	*Report*
non_null_status_implies_valid_tokenId	Verified	*An invariant helper:* *Any request with a non-NULL status has a tokenId strictly less than nextTokenId.*	*Report*
processed_requests_meet_minimum	Verified	*An invariant helper:* *Every request in the Processed state has its usdatOwed field greater than or equal to its minUsdatReceived field.*	*Report*

P-05. The withdrawal queue always holds enough USDat to cover all processed requests

Status: Verified	**High-level description** The USDat balance held by the withdrawal queue is always greater than or equal to the total USDat owed across all requests in the Processed state.	

Rule Name	Status	Description	Link to rule report
queue_solvency	Verified	*See above*	*Report*

P-06. Only the expected functions may modify usdatBalance / strcBalance

Status: Verified	**High-level description** No function other than those explicitly designated may change the usdatBalance or strcBalance state variables of StakedUSDat.	

Rule Name	Status	Description	Link to rule report
only_expected_functions_modify_usdat_balance	Verified	*No function other than deposit(), mint(), depositWithMinShares(), mintWithMaxAssets(), depositWithPermit(), mintWithPermit(), convertFromUsdat(), convertFromStrc(), and collectDust() may change usdatBalance.*	*Report*
only_expected_functions_modify_strc_balance	Verified	*No function other than convertFromUsdat(), convertFromStrc(), transferInRewards(), and burnQueuedShares() may change strcBalance.*	*Report*

P-07. Access Control for StakedUSDat and WithdrawalQueueERC721

Status: Verified	**High-level description** A paused contract or a blacklisted address cannot successfully execute any operation that the protocol restricts for that status.	

Rule Name	Status	Description	Link to rule report
blacklisted_correctness__sUSDat	Verified	*A blacklisted address cannot successfully call any function on StakedUSDat, except approve, permit, and renounceRole.*	*Report*
blacklisted_correctness__queue	Verified	*A blacklisted address cannot successfully call any function on the WithdrawalQueue, except approve, setApprovalForAll, and renounceRole.*	
paused_correctness__sUSDat	Verified	*When StakedUSDat is paused, a standard user can't call any function except the claim-functions, permit/approve, and renounceRole.*	
paused_correctness__sUSDat__claim_funcs	Verified	*When StakedUSDat is paused, calling claim() or claimBatch() cannot change the caller's sUSDat share balance.*	
paused_correctness__queue	Verified	*When the WithdrawalQueue is paused, a standard user can't call any function except approve, setApprovalForAll, and renounceRole.*	
is_user_callable__sUSDat__correctness	Verified	*Validates that the above rules cover every non-view function a standard user can invoke on StakedUSDat.*	
is_user_callable__queue__correctness	Verified	*Validates that the above rules cover every non-view function a standard user can invoke on WithdrawalQueueERC721.*	

P-08. totalAssets() is unchanged after setVestingPeriod()

Status: Verified	**High-level description** Calling setVestingPeriod() does not alter the total assets held by the StakedUSDat contract.	

Rule Name	Status	Description	Link to rule report
total_assets_unchanged_after_set_vesting_period	Verified	*See above*	*Report*

USDat Properties

Trust Assumptions

1. We assume that loop_iter is 3. That means that we only unroll Solidity loops up-to 3 times.
2. For property-10 we assume that there are at most 2 assets.

P-09. USDat is always backed

Status: Verified	**High-level description** The sum of mToken held by USDat and its total virtual assets always covers the total supply.

Rule Name	Status	Description	Link to rule report
usdatAlwaysBacked	Verified	*See above*	*Report*

P-10. Virtual accounting is consistent and conservative

Status: Verified	**High-level description** The contract's internal accounting variables are mutually consistent, and the virtual balance recorded for each asset never exceeds the actual ERC20 balance held by USDat.

Rule Name	Status	Description	Link to rule report
internal_accounting_consistency	Verified	*The contract's totalAssets() counter never exceeds the normalized sum of all individual per-asset virtual balances.* *Limitation: this rule was checked for up to 2 assets.*	*Report*

virtual_accounting_integrity	Verified	*The virtual balance recorded internally for each asset never exceeds the real ERC20 balance of that asset held by USDat.*	

P-11. Whitelist, freeze, and pause configurations are never ignored

Status: Verified	**High-level description** A non-whitelisted, frozen, or paused actor cannot successfully execute any operation that the protocol restricts for that status, and only the designated role-holder can change each configuration.

Rule Name	Status	Description	Link to rule report
whitelist_blocks_operation	Verified	*When the whitelist is enabled, any M-path operation (swapInM() / swapOutM()) involving a non-whitelisted caller or recipient reverts.*	Report
whitelist_JMI_bypass	Verified	*The JMI-path swap (swap()) reverts when the caller or recipient is not whitelisted.*	
force_transfer_requires_frozen	Verified	*Calling forceTransfer() on an account that is not frozen reverts.*	
only_forced_transfer_manager_can_force_transfer	Verified	*forceTransfer() reverts for any caller that does not hold FORCED_TRANSFER_MANAGER_ROLE.*	
only_whitelist_manager_can_manage_whitelist	Verified	*Every whitelist management function reverts for any caller that does not hold WHITELIST_MANAGER_ROLE.*	

freeze_blocks_operation	Verified	*Any token operation (swapInM(), swap(), swapOutM(), transfer(), transferFrom()) involving a frozen caller, sender, or recipient reverts.*	
only_freeze_manager_can_freeze	Verified	*Every freeze and unfreeze function reverts for any caller that does not hold FREEZE_MANAGER_ROLE.*	
pause_blocks_operation	Verified	*When the contract is paused, every token operation (swapInM(), swap(), swapOutM(), transfer(), transferFrom()) reverts.*	
only_pauser_can_pause	Verified	*The pause and unpause functions revert for any caller that does not hold PAUSER_ROLE.*	

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.